

**ТЕХНИЧЕСКАЯ ДОКУМЕНТАЦИЯ**  
**по проекту**  
**"Conveyor Vision"**

**Студент: Ахметов Нияз Маликович**

**Курс: Основы программирования на Python с применением Искусственного  
Интеллекта**

**Дата: 2025 г.**

Данная документация описывает процесс создания, тестирования и подготовки к коммерческому использованию системы Conveyor Vision, предназначенной для автоматического выявления повреждений на конвейерных лентах.

В документе подробно представлены:

- цели проекта;
- архитектура решения;
- этапы подготовки данных и обучения модели YOLOv8;
- описание инференса и визуализации результатов;
- состав вспомогательных модулей;
- требования к запуску и инструкции по установке;
- план по коммерческой реализации решения.

Документация подготовлена как часть дипломной работы по курсу «Основы программирования на Python с применением Искусственного Интеллекта» и может быть использована в качестве технической базы при внедрении системы в производственной среде.

## Назначение проекта

Проект **Conveyor Vision** представляет собой программное обеспечение для автоматического выявления повреждений на конвейерной ленте с использованием технологий компьютерного зрения. Приложение позволяет пользователю загружать изображения, автоматически выполнять детекцию повреждённых участков, визуализировать результаты и сохранять журнал обработки.

**Цель:** минимизация человеческого фактора и оперативное обнаружение повреждений ленты для предотвращения внеплановых остановок оборудования и повышения эффективности технического обслуживания.

Данный проект является **дипломной работой по курсу "Основы программирования на языке Python с применением Искусственного Интеллекта"** и предназначен для демонстрации полученных знаний и навыков не только в применении инструментов ИИ, но и в решении реальных производственных задач, направленных на создание коммерчески жизнеспособных продуктов.

На текущем этапе проект реализован как **MVP (Minimum Viable Product)**, полностью работоспособный в локальной среде. После завершения учебы планируется запуск продукта в коммерческую эксплуатацию на реальном производстве с возможностью масштабирования и интеграции в существующие системы технического обслуживания и мониторинга.

## 1 Путь реализации решения

### Этапы создания и настройки модели:

#### 1. Сбор и подготовка данных

- Были собраны изображения участков конвейерной ленты с различными типами состояния — повреждённые (Bad) и неповреждённые (Good).
- Разметка изображений проводилась на платформе Roboflow, где каждому объекту был присвоен соответствующий класс.
- Аннотации сохранялись в формате COCO, с разбивкой на три выборки: обучающую (train), валидационную (valid) и тестовую (test).
- Классы размечены как:
  - 0 — Bad
  - 1 — Good

## Обучение модели YOLOv8

- Использована лёгкая версия модели YOLOv8n.pt из библиотеки **Ultralytics**.
- Конфигурация: 50 эпох, размер изображений 640×640, батч 16.
- Для обучения использован скрипт train\_yolo.py, в котором явно задавались пути к датасету и параметры модели.
- Обучение прошло успешно: по результатам последней эпохи модель показала следующие метрики на валидационной выборке:
  - mAP50: **0.932**,
  - mAP50-95: **0.812**,
  - Precision: **0.888**,
  - Recall: **0.927**.
- Вес модели best.pt был сохранён без оптимизатора и готов к использованию в приложении.

### 2. Разработка пользовательского интерфейса

- Создан модуль app.py на фреймворке Streamlit с загрузкой модели best.pt.
- Добавлены функции загрузки изображения, запуск инференса, визуализация результатов с помощью библиотеки PIL, сохранение изображения, отображение метрик и журналов обработки.
- Интерфейс поддерживает:
  - Фильтрацию по классам (Good/Bad)
  - Настройку порога уверенности (confidence threshold)
  - Скачивание результата и журнала в формате CSV

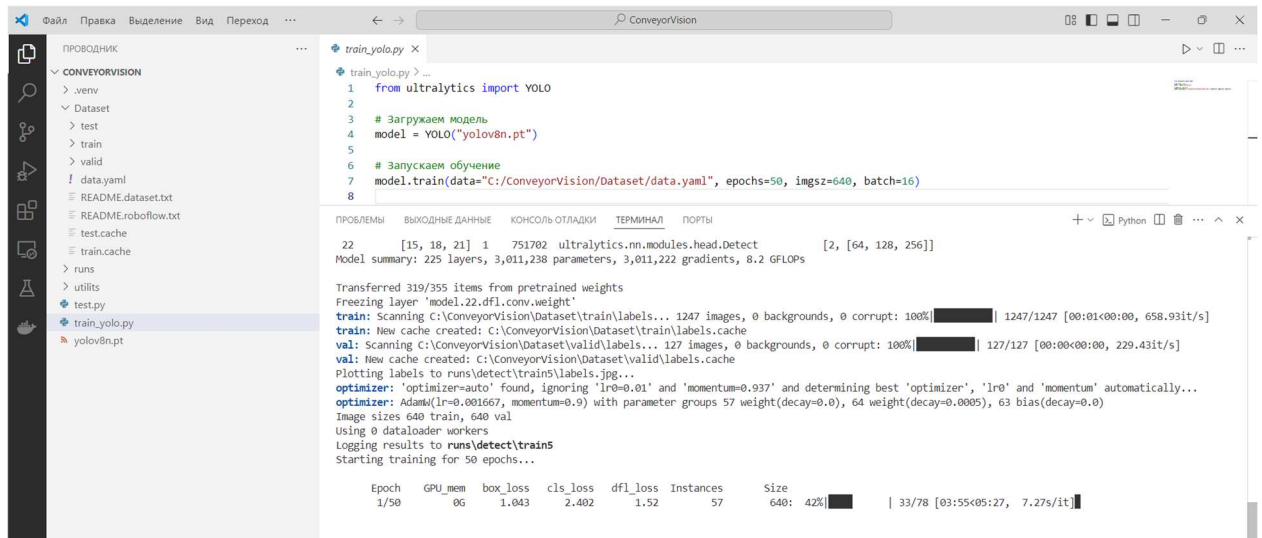
### 3. Создание модуля main.py

- Выполняет функции точки входа и координации между частями системы.
- Поддерживает интеграцию будущих модулей (например, обработка видео, REST API, автоматическое логирование).

### 4. Информация о модели

- Модель содержит 2 класса (Bad, Good), 3 006 038 параметров, среднее время инференса — 118 мс на изображение (CPU).

- Подробности реализованы в модуле model\_information.py.



**Рисунок 1. Процесс обучения модели YOLOv8 в среде VS Code.**

Отображены параметры обучения: 50 эпох, размер изображения 640×640, размер батча 16. Обучение проводится на размеченном датасете Conveyor Vision с использованием предобученной модели yolov8n.pt. Показаны данные о слоях модели, количестве изображений в выборке и метриках.

## Модули автоматизации и подготовки данных (в порядке применения)

### 1. import\_dataset.py — Загрузка датасета из Roboflow

**Назначение:** автоматическая загрузка размеченного датасета из облачного проекта Roboflow.

#### Библиотека:

- roboflow — официальная библиотека Roboflow API.

#### 🔧 Как работает:

- Авторизуется по api\_key пользователя
- Подключается к проекту по имени (conveyer-belt-vz1tr-riyjb)
- Загружает указанную версию датасета (например, v2)
- Сохраняет датасет в папку C:/ConveyorVision/dataset

```
rf = roboflow.Roboflow(api_key="...") # авторизация
```

```
project = rf.workspace().project("имя_проекта")
```

```
dataset = project.version(2).download(location="папка")
```

## 2. delete\_class.py — Очистка датасета от лишнего класса

**Назначение:** удаление ненужного класса из аннотаций COCO.

Например: удаление class\_id = 0 («Good») для задач, где важна только детекция повреждений (Bad).

**Библиотека:**

- json — для чтения и записи COCO-файла

**Как работает:**

- Загружает файл \_annotations.coco.json
- Фильтрует categories, исключая категорию с id = 0
- Сохраняет обновлённый JSON обратно

```
data["categories"] = [cat for cat in data["categories"] if cat["id"] != 0]
```

## 3. conveyor\_vision\_roboflow.py — Проверка загрузки и структуры модели

**Назначение:** тестовая загрузка модели YOLOv8 и вывод её структуры.

**Библиотека:**

- ultralytics — фреймворк для работы с YOLOv8

**Как работает:**

- Загружает базовую модель yolov8n.pt
- Выводит её описание (архитектура, параметры)

```
from ultralytics import YOLO
```

```
model = YOLO("yolov8n.pt")
```

```
print(model)
```

Используется на стадии инициализации проекта и проверки корректности модели.

## 4. automat.py — Визуализация размеченных изображений из COCO

**Назначение:** интерактивный просмотр размеченных изображений с аннотациями (bounding boxes и названия классов).

**Библиотеки:**

- cv2 (OpenCV) — для чтения изображений и отрисовки рамок
- json — загрузка аннотаций

- matplotlib.pyplot — отображение изображения

#### Как работает:

- Загружает COCO-аннотацию
- Проходит по изображениям поочерёдно
- Для каждого — рисует рамки, классы, выводит инфо в консоль
- Ждёт нажатия Enter → показывает следующее изображение

```
cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
```

```
cv2.putText(image, label, (x, y - 10), ...)
```

```
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
```

Полезен на стадии **верификации датасета перед обучением**.

#### Итоговый порядок использования:

| Этап | Модуль                      | Цель                                         |
|------|-----------------------------|----------------------------------------------|
| 1    | import_dataset.py           | Загрузка датасета из Roboflow                |
| 2    | delete_class.py             | Удаление лишних классов (например, Good)     |
| 3    | conveyor_vision_roboflow.py | Проверка модели YOLOv8                       |
| 4    | automat.py                  | Проверка аннотаций и отображение изображений |

#### Этапы создания и настройки модели:

##### 1. Сбор и подготовка данных

- Данные были собраны с производственного объекта, где фиксировались изображения ленты с нормальным и повреждённым состоянием.
- Для аннотирования использована платформа [Roboflow](#), через которую вручную размечены объекты двух классов: Good (норма) и Bad (повреждения).
- Аннотации выгружались в формате COCO и дополнительно конвертировались в формат YOLO при помощи скрипта `convert_coco_to_yolo.py`.

- Структура папок включала разбиение на train, valid, test, а для каждой выборки создавались .txt-файлы с аннотациями в YOLO-формате.
- Скрипты class.py, automat\_class.py, delete\_class.py и import\_dataset.py использовались для:
  - проверки классов и изображений по категориям
  - фильтрации и удаления лишних классов
  - загрузки датасета из Roboflow по API

## 2 Обучение модели YOLOv8

- Для обучения выбрана компактная и быстрая модель YOLOv8n.pt, поставляемая через библиотеку **Ultralytics**.
- Основной скрипт обучения — train\_yolo.py:
- `from ultralytics import YOLO`
- `model = YOLO("yolov8n.pt")`

`model.train(data="C:/ConveyorVision/Dataset/data.yaml", epochs=50, imgsz=640, batch=16)`

- Обучение проводилось на локальном ПК под управлением Windows 10, в среде VS Code с установленными библиотеками ultralytics, torch, opencv-python, numpy и др.
- В файле data.yaml указаны пути к тренировочной и валидационной выборкам, а также описание классов.
- Обучение длилось 50 эпох с размером изображения 640 пикселей и батчем 16. Модель автоматически сохраняла веса по мере улучшения метрик.
- По результатам обучения был сформирован файл best.pt (лучшая версия модели), который используется для инференса в приложении.
- В модуле model\_information.py описаны ключевые параметры итоговой модели:
  - Название модели: DetectionModel
  - Количество классов: 2
  - Имена классов: 0 — Bad, 1 — Good
  - Количество параметров: 3 006 038
- Среднее время инференса: ~118 мс



- Расширенные описания классов включают назначение каждого класса на практике.
- Обучение показало хорошие метрики (mAP, precision, recall) и готовность модели к интеграции в реальное приложение.
- Использована предобученная модель yolov8n.pt как базовая.
- Обучение проводилось с помощью скрипта train\_yolo.py, в котором заданы:
  - путь к конфигурационному файлу data.yaml
  - 50 эпох обучения
  - размер изображений 640 пикселей
  - батч 16 изображений
- Модель обучалась на локальной машине в среде VS Code.
- По окончании обучения сохранён файл best.pt, который используется в основном приложении для инференса.

## **2. Разработка пользовательского интерфейса**

- Приложение разработано с использованием Streamlit (app.py/main.py).
- Используется библиотека PIL для отрисовки результатов.
- Интерфейс реализует:
  - загрузку изображения
  - запуск модели и отображение результатов
  - настройку порога уверенности (threshold)
  - фильтрацию по классам
  - скачивание результата
  - сохранение журналов обработки и миниатюр результатов

## **3. Модули автоматизации и подготовки данных**

- automat.py — визуализация размеченных данных из COCO
- conveyor\_vision\_roboflow.py — проверка загрузки модели
- delete\_class.py — удаление лишнего класса из аннотаций
- import\_dataset.py — загрузка датасета из Roboflow по API

## **4. Основные параметры модели**

- Классы: 2 (0: Bad, 1: Good)
- Параметры модели: 3 006 038
- Время инференса: ~118 мс на изображение
- Источник: модуль model\_information.py

### Анализ модели best.pt и её структура

В процессе обучения модели YOLOv8 была сохранена лучшая версия модели под именем best.pt, которая впоследствии используется для инференса в приложении. Данный файл содержит всё необходимое для предсказаний: веса, архитектуру модели, параметры обучения, метрики и список классов. Ниже приводится детальный анализ содержимого данного файла, полученный с помощью библиотеки Ultralytics и PyTorch.

### Общая информация о модели

Модель была загружена с помощью класса YOLO из библиотеки ultralytics:

```
from ultralytics import YOLO

model = YOLO("runs/detect/train5/weights/best.pt")

model.info()
```

### Результат:

Model summary: 129 layers, 3,011,238 parameters, 0 gradients, 8.2 GFLOPs

- **Число слоёв:** 129
- **Количество параметров:** 3,011,238
- **Объём вычислений (FLOPs):** 8.2 GFLOPs
- **Градиенты:** отсутствуют (модель используется только для инференса)

Модель — лёгкая версия **YOLOv8n**, специально оптимизированная для работы на слабом оборудовании (например, Jetson Nano).

### Классы, на которых обучалась модель

```
print(model.names)
```

Результат:

```
{0: 'Bad', 1: 'Good'}
```

Таким образом, модель обучалась на 2 классах:

- 0 — Bad — участки конвейерной ленты с повреждениями
- 1 — Good — исправные участки без видимых дефектов

### Вскрытие содержимого best.pt через PyTorch

```
import torch
```

```
checkpoint = torch.load("runs/detect/train5/weights/best.pt", map_location="cpu")
```

```
print(checkpoint.keys())
```

Результат:

```
['date', 'version', 'license', 'docs', 'epoch', 'best_fitness',  
'model', 'ema', 'updates', 'optimizer',  
'train_args', 'train_metrics', 'train_results']
```

Описание ключей:

| Ключ          | Значение / Назначение                              |
|---------------|----------------------------------------------------|
| model         | Объект модели YOLO с архитектурой и весами         |
| ema           | Экспоненциально усреднённая версия модели          |
| train_args    | Аргументы обучения (эпохи, batch, lr и др.)        |
| train_metrics | Итоговые метрики после завершения обучения         |
| optimizer     | Состояние оптимизатора AdamW                       |
| train_results | История обучения по эпохам                         |
| best_fitness  | Значение функции пригодности модели                |
| epoch         | Номер эпохи, на которой достигнут лучший результат |

### Архитектура модели

Модель состоит из типичных компонентов YOLOv8:

- Сверточные блоки Conv
- Блоки C2f и Bottleneck — отвечают за извлечение признаков
- SPPF — Spatial Pyramid Pooling Fast

- Upsample, Concat — объединение признаков на разных масштабах
- Финальный Detect блок с головой для трёх уровней: P3, P4, P5
- DFL — Distribution Focal Loss (уточнение координат)

Это подтверждает, что модель является полной реализацией YOLOv8n с тремя уровнями выходов.

### Примечание

Файл best.pt можно использовать:

- в инференсе (Streamlit, скрипты)
- для повторного обучения (YOLO("best.pt").train(...))
- для анализа и визуализации метрик (train\_metrics, train\_results)
- как часть системы встраиваемого ИИ

### Вывод

Модель best.pt — это компактное, но мощное решение, подготовленное специально под задачу обнаружения повреждений конвейерной ленты. Она обучена на кастомных данных, поддерживает детекцию двух классов, обладает невысокой сложностью (8.2 GFLOPs), что делает её подходящей для использования на маломощных устройствах, включая Jetson Nano. Её структура соответствует спецификации YOLOv8 и включает все ключевые компоненты современной CNN-архитектуры.

## 3 Процесс инференса (распознавания) модели

Инференс — это применение заранее обученной модели YOLOv8 к новому изображению с целью обнаружения объектов заданных классов:

- 0 — Bad (повреждение)
- 1 — Good (исправное состояние)

## Алгоритм работы модели на этапе инференса

### 1. Загрузка изображения

- Пользователь выбирает файл .jpg, .png или .jpeg через интерфейс Streamlit (st.file\_uploader)

- Изображение преобразуется в формат, совместимый с PIL и YOLO

## 2. Настройка параметров пользователем

- Слайдер позволяет установить порог уверенности модели (threshold, по умолчанию 0.5)
- Чекбоксы позволяют выбрать, какие классы отображать (Bad, Good)

## 3. Вызов модели YOLOv8

### 4. results = model(image)

- Загруженная модель (YOLO(MODEL\_PATH)) применяет инференс
- Полученные объекты фильтруются по confidence и class\_id

### 5. Фильтрация результатов В функции detect\_objects() отбрасываются все предсказания, у которых:

- Уверенность (confidence) ниже заданного порога
- Класс не входит в список выбранных пользователем

### 6. filtered\_boxes = [box for box in result.boxes if box.conf[0].item() >= threshold and int(box.cls[0].item()) in classes]

### 7. Визуализация результата В функции draw\_results():

- Каждому объекту отрисовывается прямоугольник (cv2.rectangle)
- Подпись над рамкой указывает имя класса и уверенность
- Цвет рамки зависит от класса:
  - Bad — красный
  - Good — зелёный

## 8. Оповещение пользователя

- Если обнаружено хотя бы одно повреждение (class\_id == 0), Streamlit показывает st.error("🚨 Обнаружено повреждение")
- Если только нормальные участки — st.success("🟢 Лента в нормальном состоянии")

## 9. Сохранение результата

- Обработанное изображение сохраняется во временный PNG-файл через tempfile
- Пользователю предлагается скачать результат (st.download\_button)

## 10. Ведение журнала

- Каждое событие логируется в `st.session_state.event_log`
- Журнал включает:
  - Время
  - Название изображения
  - Результат (True / False — наличие повреждений)
- Кнопка «Показать журнал событий» выводит таблицу в интерфейсе

## 11. Информация о модели

- Модуль `model_information.py` предоставляет информацию о модели:
  - Название модели
  - Кол-во классов
  - Размер модели, время инференса, описание классов и т.д.

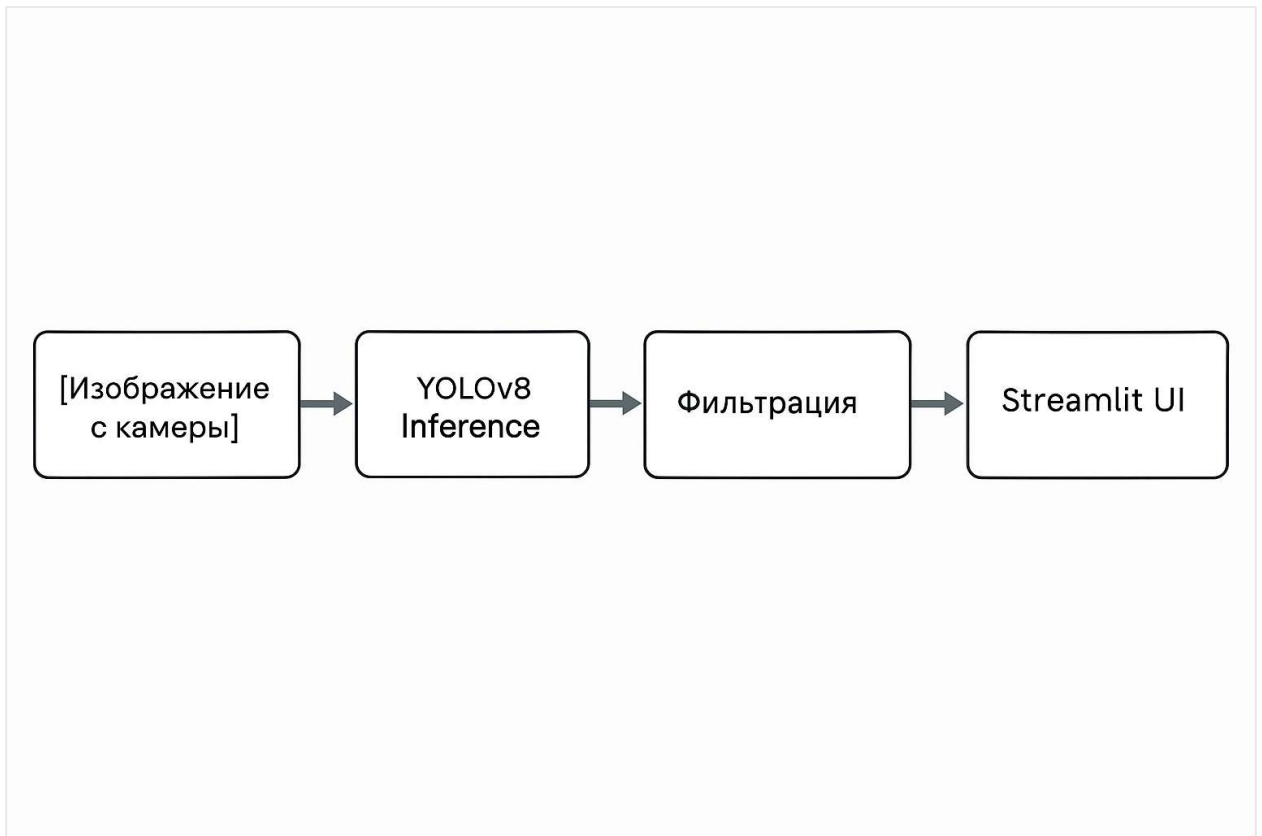


Рисунок 2. Блок-схема обработки изображения: от камеры до интерфейса пользователя.

## 4 Основные компоненты инференса

| Компонент        | Назначение                                |
|------------------|-------------------------------------------|
| YOLO(model_path) | Загрузка обученной модели                 |
| model(image)     | Получение предсказаний                    |
| detect_objects() | Фильтрация по порогу и классам            |
| draw_results()   | Отрисовка прямоугольников и классов       |
| cv2, PIL, numpy  | Обработка изображения                     |
| Streamlit        | Интерфейс, ввод/вывод, управление сессией |
| ModelInfo()      | Вывод информации о модели                 |

### Инструкция по установке и запуску

Данный раздел предназначен для пользователей, желающих развернуть локальную версию проекта **Conveyor Vision** на своей машине.

#### Требования:

- Python 3.10+
- pip
- Git (опционально)
- Рекомендуется использовать виртуальное окружение

#### Установка:

1. **Клонируйте репозиторий** (если используется Git):
2. `git clone https://github.com/NiyazAkhmetov85/Conveyor_Vision.git`
3. `cd Conveyor_Vision`
4. **Создайте виртуальное окружение (опционально):**
5. `python -m venv .venv`

6. `.venv\Scripts\activate` (Windows) или `source .venv/bin/activate` (Linux)
7. **Установите зависимости:**
8. `pip install -r requirements.txt`
9. **Запустите приложение:**
10. `streamlit run main.py`

### Доступ:

После запуска в браузере откроется:

`http://localhost:8501`

### Примечание:

- Для работы модели используется YOLOv8 (best.pt), файл должен быть расположен в корне проекта.
- Все результаты сохраняются в папку `results/`, журналы событий ведутся в `st.session_state`.

### План подготовки к коммерческой эксплуатации

Для вывода проекта **Conveyor Vision** из стадии MVP в промышленную эксплуатацию запланированы следующие этапы:

#### 1. Завершение отладки и внутреннего тестирования

- Финальная отладка интерфейса, логики фильтрации и сохранения результатов
- Модульное тестирование всех функций и сохранение логов работы
- Расширение возможностей загрузки: поддержка анализа изображений из папки и пакетной обработки

#### 2. Расширенное тестирование на промышленном объекте

- Развёртывание системы на тестовом участке действующего производства
- Оценка качества инференса в реальных условиях освещения и пылеобразования
- Оценка производительности на слабом оборудовании (без GPU)

#### 3. Оптимизация и подготовка продукта к распространению

- Сбор обратной связи от операторов и инженеров



- Оптимизация модели (возможный переход на ONNX/OpenVINO)
- Вынесение конфигураций в отдельный YAML/JSON-файл для облегчения настройки
- Автологирование в SQLite или CSV

#### 4. Подготовка установочного пакета и инструкции

- Формирование стабильной версии под Windows/Linux
- Создание .exe-установщика (через PyInstaller)
- Подготовка пользовательской инструкции по установке и применению

#### 5. Интеграция ИИ-агента для сопровождения

- Проработка концепции ИИ-ассистента, встроенного в интерфейс (подсказки, рекомендации, логика анализа)
- Возможная реализация на базе платформы Parlant или собственных решений

#### 6. Коммерциализация и вывод на рынок

- Подготовка презентационных материалов (PDF, видео, сайт)
- Регистрация авторских прав и домена
- Поиск первых пилотных клиентов
- Выход на малый бизнес и подрядчиков на предприятиях горнодобывающей и перерабатывающей промышленности

### Структура проекта Conveyor\_Vision

Conveyor\_Vision/

```
└─ app.py          # Основной интерфейс на Streamlit
└─ main.py         # Точка входа, объединяющая интерфейс и логику
└─ model_information.py  # Модуль с описанием параметров модели
└─ train_yolo.py   # Скрипт для обучения модели YOLOv8
└─ requirements.txt # Зависимости проекта (Streamlit, Ultralytics и др.)
└─ README.md       # Краткое описание проекта и инструкция запуска
└─ .gitignore      # Исключаем лишние файлы из Git
|
```

└─ best.pt           # Обученная модель YOLOv8 (если размещается отдельно —  
можно дать ссылку на загрузку)

|

└─ results/           # Сохраняемые изображения с результатами инференса

|   └─ result\_\*.png    # Файлы результата

|

└─ dataset/           # Папка с датасетом (может быть исключена из GitHub)

|   └─ data.yaml       # Конфигурация датасета

|

└─ utils/            # Вспомогательные скрипты (можно объединить в папку)

|   └─ convert\_coco\_to\_yolo.py

|   └─ import\_dataset.py

|   └─ delete\_class.py

|   └─ automat.py

|   └─ conveyor\_vision\_roboflow.py

|   └─ class.py

|   └─ automat\_class.py

---

## Файлы для загрузки в GitHub

### Обязательно:

- app.py
- main.py
- model\_information.py
- requirements.txt
- train\_yolo.py
- README.md
- .gitignore

### По желанию (в utils/):

- convert\_coco\_to\_yolo.py

- import\_dataset.py
- automat.py (проверка аннотаций)
- delete\_class.py
- conveyor\_vision\_roboflow.py

#### Исключить из репозитория:

- best.pt — лучше хранить на [Hugging Face](#) или Google Drive и указать ссылку
- results/, dataset/ — можно игнорировать через .gitignore

#### Перечень необходимого оборудования и ПО:

##### 1. Аппаратное обеспечение:

- **NVIDIA Jetson Nano Developer Kit 4GB:**
  - **Описание:** Компактный и энергоэффективный компьютер для разработки и прототипирования проектов в области искусственного интеллекта.
  - **Стоимость:** 175 000 KZT (примерно 375 USD).
  - **Доставка по РК:** Включена в стоимость.
- **Камера Raspberry Pi Camera Module V2:**
  - **Описание:** Камера с разрешением 8 МП, совместимая с Jetson Nano.
  - **Стоимость:** 15 000 KZT (примерно 32 USD).
- **Аксессуары:**
  - **Карта памяти microSD 32GB:** 5 000 KZT (примерно 11 USD).
  - **Блок питания 5V 4A:** 4 000 KZT (примерно 9 USD).
  - **Итого:** 199 000 KZT (примерно 427 USD).

##### 2. Программное обеспечение:

- **Операционная система:** Ubuntu 18.04 для Jetson Nano (бесплатно).
- **Платформа разработки:** NVIDIA JetPack SDK, включающий необходимые библиотеки для разработки ИИ-приложений (бесплатно).
- **Фреймворки и библиотеки:**
  - **TensorFlow или PyTorch:** Для разработки и обучения моделей (бесплатно).

- **OpenCV:** Для обработки изображений и видео (бесплатно).
- **Streamlit:** Для создания веб-интерфейса приложения (бесплатно).

**Общая стоимость:**

- **Оборудование:** 199 000 KZT (примерно 427 USD).
- **Программное обеспечение:** Бесплатно.

**Итого:** 199 000 KZT (примерно 427 USD).

Дополнительно выполнена оценка затрат на оборудование для создания полевого, автономного комплекта, включающего резервное питание, защиту от внешней среды, прочную стойку и промышленный корпус. С учётом доставки, таможенных пошлин и дополнительного резерва, примерная сумма требуемых инвестиций составляет около 1000 долларов США.

**2. Программное обеспечение:**

- **Операционная система:** Ubuntu 18.04 для Jetson Nano (бесплатно).
- **Платформа разработки:** NVIDIA JetPack SDK, включающий необходимые библиотеки для разработки ИИ-приложений (бесплатно).
- **Фреймворки и библиотеки:**
  - **TensorFlow или PyTorch:** Для разработки и обучения моделей (бесплатно).
  - **OpenCV:** Для обработки изображений и видео (бесплатно).
  - **Streamlit:** Для создания веб-интерфейса приложения (бесплатно).

## **5 Перспективы развития**

Проект "Conveyor Vision" направлен не только на решение задачи детекции повреждений конвейерных лент, но и на накопление опыта в применении технологий машинного обучения в промышленности. Полученные знания и наработки могут быть адаптированы для решения широкого спектра задач, таких как мониторинг оборудования, предиктивное обслуживание и автоматизация производственных процессов. Это открывает возможности для расширения бизнеса и предоставления инновационных решений клиентам, что принесет выгоду как инвесторам, так и предприятиям-заказчикам.

## **Заключение**

Проект Conveyor Vision успешно реализован как MVP-решение для автоматического выявления повреждений на конвейерной ленте с применением модели YOLOv8. Разработана рабочая локальная версия приложения с интуитивным интерфейсом, реализован модуль инференса и сохранения результатов. Проект имеет потенциал к масштабированию и коммерческому использованию в реальных производственных условиях. Планируется дальнейшее развитие в направлении промышленного прототипа и интеграции с оборудованием на базе NVIDIA Jetson Nano.